Intro to Data Science Project

2025-10-18

1)Table displaying summary statistics for the gene you included in your check-ins as well as one other gene.

```
setwd("/Users/katerinasmith/Desktop/HDS/Intro to Data Science/Project")
# using read.csv from lecture notes
counts <- read.csv(file = "counts.csv", stringsAsFactors = FALSE)

#head(counts)

#First gene
#ESR1 Gene, estrogen receptor 1
#https://useast.ensembl.org/Homo_sapiens/Gene/Summary?db=core;g=ENSG00000091831;r=6:151651284-152129619
first_gene <- "ENSG00000091831.24"
first_gene_name <- "ESR1"

#filter for the fist gene and rename first column for gene name
library(data.table)
my_gene <- counts[counts[, 1] %like% first_gene, ]
#print(my_gene)


names(my_gene)[1] <- "gene_name"
#print(my_gene)

#Use pivot_longer function from lecture to pivot data for analysis, different method learned here: #htt
library(tidyr)
my_gene.dt <- my_gene %>%
  pivot_longer(-gene_name, names_to = "sample", values_to = "count_value")

#head(my_gene.dt)

#Summary statistics for first gene
summary_stats_table <- data.frame(
  Statistics = c("Min", "1st Qu", "Median", "Mean", "3rd Qu", "Max", "SD"),
  Value = c(
    min(my_gene.dt$count_value),
    quantile(my_gene.dt$count_value, 0.25),
    median(my_gene.dt$count_value),
    mean(my_gene.dt$count_value),
    quantile(my_gene.dt$count_value, 0.75),
    max(my_gene.dt$count_value),
    sd(my_gene.dt$count_value)
  )
)

#round values
```

```r
summary_stats_table$Value <- round(summary_stats_table$Value, 2)

print(paste("Summary Statistics for", first_gene_name, "Gene"))
```

```
## [1] "Summary Statistics for ESR1 Gene"
```

```r
print (summary_stats_table)
```

```
##   Statistics     Value
## 1        Min     14.00
## 2     1st Qu   3440.00
## 3     Median  17561.00
## 4       Mean  31442.22
## 5     3rd Qu  43682.50
## 6        Max 270977.00
## 7         SD  38261.25
```

```r
#save as image using example here: https://stackoverflow.com/questions/23365096/r-save-table-as-image
library(gridExtra)

png("summary_stats_table", width = 600, height = 400)
grid.table(summary_stats_table)
dev.off()
```

```
## pdf
##   2
```

```r
#Second gene
#PGR progesterone receptor [Source:HGNC Symbol;Acc:HGNC:8910]
#https://useast.ensembl.org/Homo_sapiens/Gene/Summary?db=core;g=ENSG00000082175;r=11:101029624-10112981
second_gene <- "ENSG00000082175.15"
second_gene_name <- "PGR"

#filter for second gene and rename first column for gene name
library(data.table)
my_sec_gene <- counts[counts[, 1] %like% second_gene, ]
#print(my_sec_gene)

names(my_sec_gene)[1] <- "gene_name"
#print(my_sec_gene)

#Use pivot_longer function from lecture to pivot data for analysis, different method learned here: #htt
my_sec_gene.dt <- my_sec_gene %>%
  pivot_longer(-gene_name, names_to = "sample", values_to = "count_value")

#Summary statistics for second gene in a table format
sec_summary_stats_table <- data.frame(
  Statistics = c("Min", "1st Qu", "Median", "Mean", "3rd Qu", "Max", "SD"),
  Value = c(
    min(my_sec_gene.dt$count_value),
    quantile(my_sec_gene.dt$count_value, 0.25),
    median(my_sec_gene.dt$count_value),
    mean(my_sec_gene.dt$count_value),
    quantile(my_sec_gene.dt$count_value, 0.75),
    max(my_sec_gene.dt$count_value),
    sd(my_sec_gene.dt$count_value)
```

```
  )
)

#round values
sec_summary_stats_table$Value <- round(sec_summary_stats_table$Value, 2)

print(paste("Summary Statistics for", second_gene_name, "Gene"))
```

```
## [1] "Summary Statistics for PGR Gene"
```

```
print (sec_summary_stats_table)
```

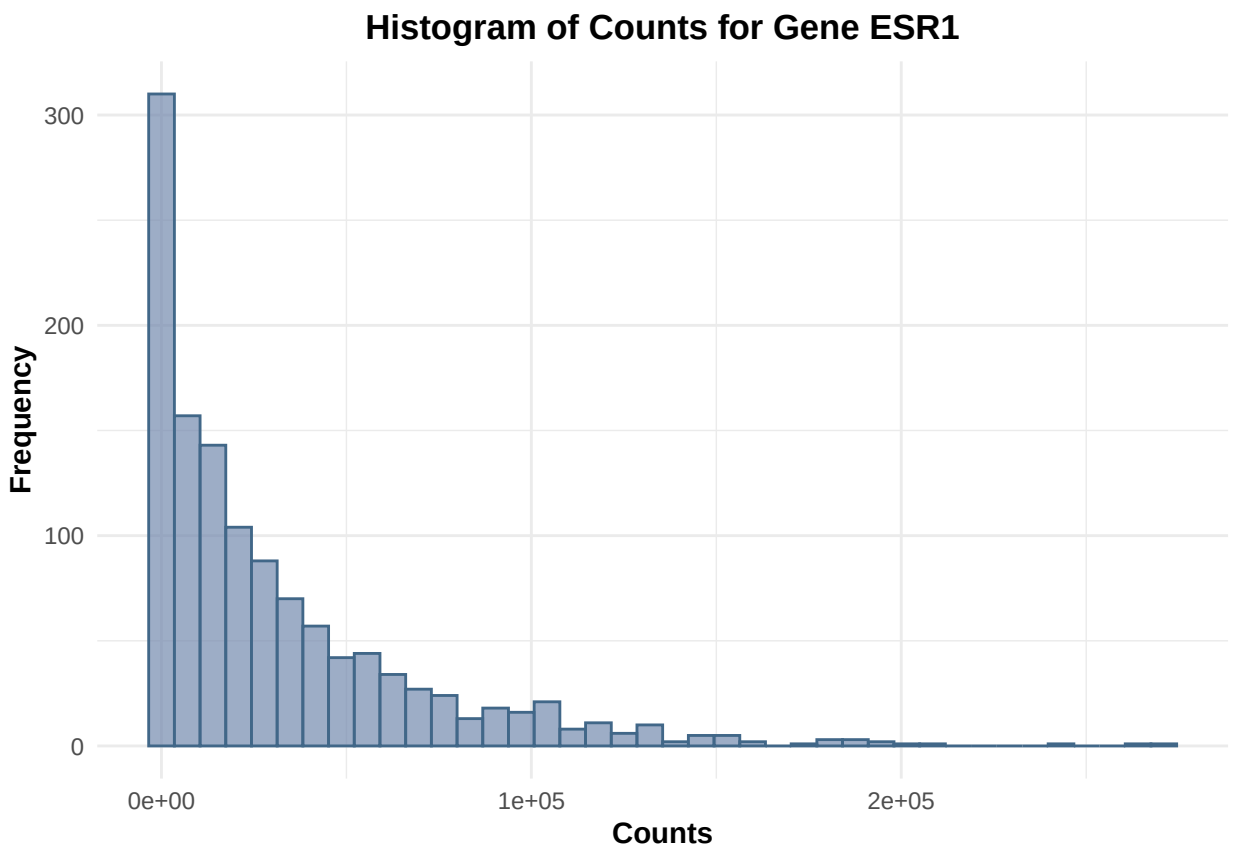
```
##   Statistics     Value
## 1        Min      1.00
## 2     1st Qu    215.00
## 3     Median   2382.00
## 4       Mean   8791.82
## 5     3rd Qu  10387.50
## 6        Max 345065.00
## 7         SD  17529.30
```

```
#save as image using example here: https://stackoverflow.com/questions/23365096/r-save-table-as-image

png("sec_summary_stats_table", width = 600, height = 400)
grid.table(sec_summary_stats_table)
dev.off()
```

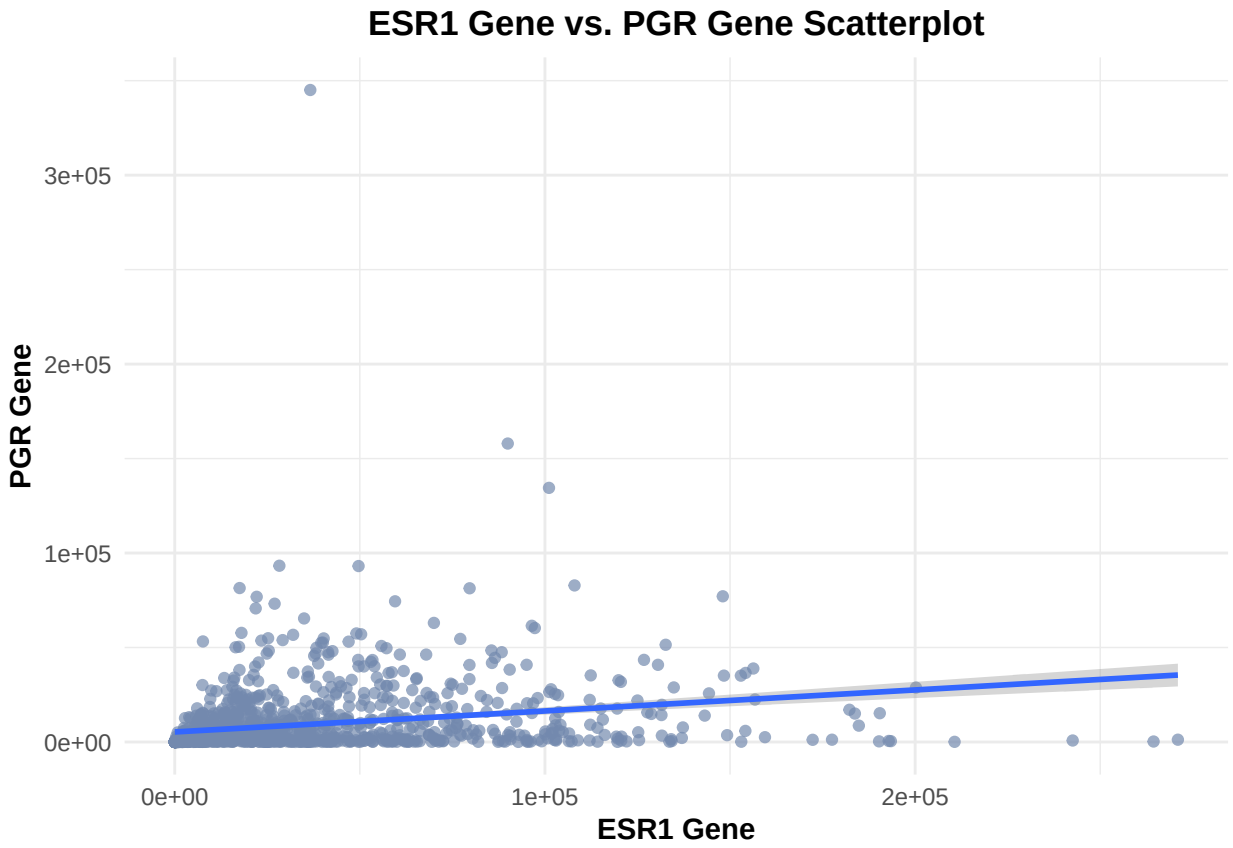
```
## pdf
##   2
```

2) Final histogram, scatter plot, and box plot from submission 1.

```
library(ggplot2)

# Histogram
#Create histogram using ggplot plot instead of base R, code referenced from lecture 3 notes
#color palette from coolors.com
ggplot(my_gene.dt, aes(x = count_value)) +
    geom_histogram(bins = 40, fill = "#7389AE", alpha = 0.7, color = "#416788") +
    labs(title = paste("Histogram of Counts for Gene", first_gene_name),
         x = "Counts",
         y = "Frequency") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
          axis.title = element_text(face = "bold"))
```




```
#Create dataframe with two genes for scatter plot
my_two_genes <- data.frame(
  first_gene = my_gene.dt$count_value,
  second_gene = my_sec_gene.dt$count_value
)

#Scatter plot for 2 genes - code referenced from lecture 3 notes
#Code to add regression line found here: https://www.geeksforgeeks.org/r-language/add-regression-line-t
ggplot(my_two_genes, aes(x = first_gene, y = second_gene)) +
  geom_point(alpha = 0.7, color = "#7389AE") +             # scatter points with some transparency
  labs(
    title = paste(first_gene_name, "Gene vs.", second_gene_name, "Gene Scatterplot"),
    x = paste(first_gene_name, "Gene"),
    y = paste(second_gene_name, "Gene")
  ) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.title = element_text(face = "bold")) +
  stat_smooth(method = "lm",
              formula = y ~ x,
              geom = "smooth")
```

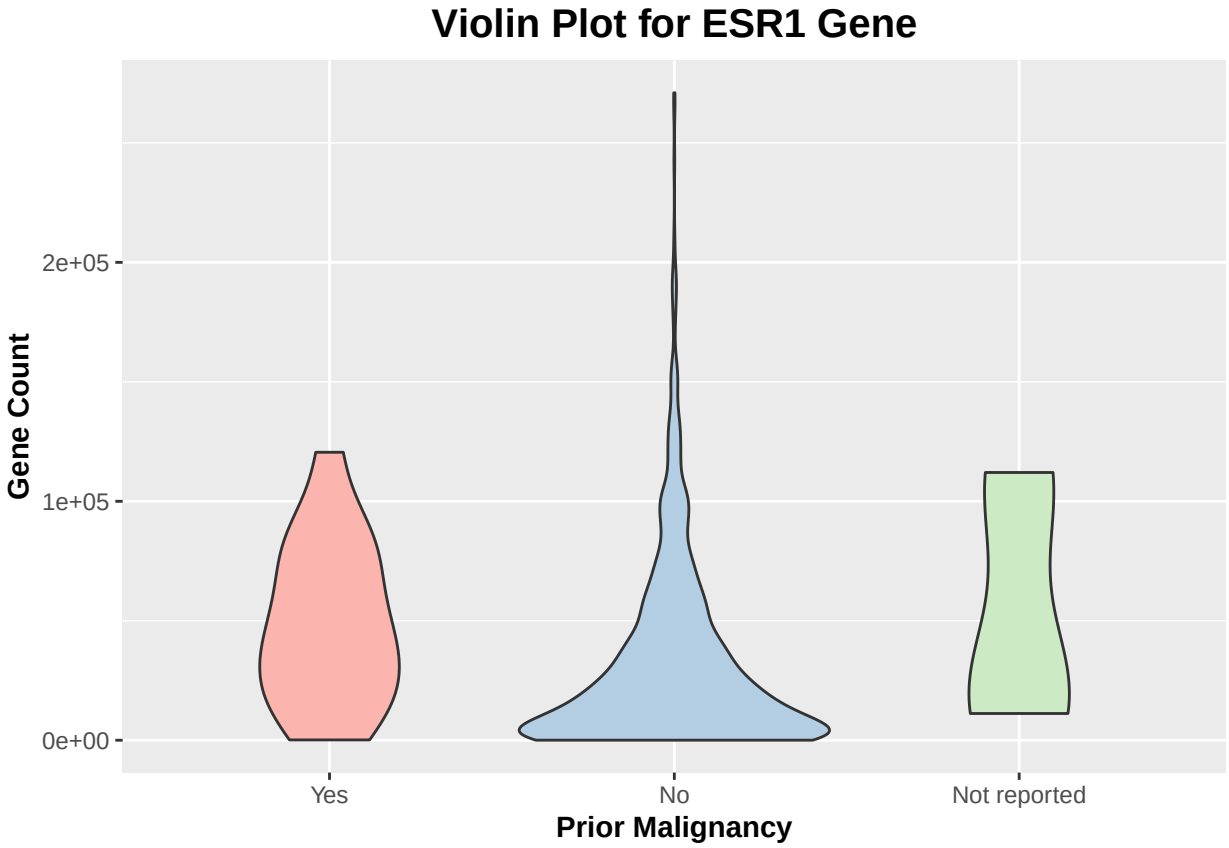



```
#Violin plot
#Get metadata
metadata <- read.csv(file = "meta_data.csv", stringsAsFactors = FALSE)
#head(metadata)

library(ggplot2)

#use gsub function to replace "." with "-" in the values of the sample column, so join #will work downs
new_gene_data <- my_gene.dt
#head(new_gene_data)

new_gene_data$sample <- gsub("\\.","-", as.character(new_gene_data$sample))
#print(new_gene_data)

#Change sample column to barcode, to match name in metadata for easy join
names(new_gene_data)[names(new_gene_data) == "sample"] <- "barcode"
#print(new_gene_data)

# Join new_gene_data for my selected gene to the metadata dataset, by the barcode column, to bring in m
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
## The following object is masked from 'package:gridExtra':
##
##     combine
```

```
## The following objects are masked from 'package:data.table':
##
##     between, first, last

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
plot_data <- new_gene_data %>%
  inner_join(select(metadata, barcode, prior_malignancy),
             by = "barcode")

#remove NAs from dataset
plot_data <- plot_data[!is.na(plot_data$prior_malignancy),]

#Convert prior_malignancy to camelcase, using str_to_sentence from here: https://stringr.tidyverse.org/
library(stringr)
plot_data$prior_malignancy <- str_to_sentence(plot_data$prior_malignancy)

#order values in prior malignancy
plot_data$prior_malignancy <- factor(plot_data$prior_malignancy, levels = c("Yes", "No", "Not reported"))

#head(plot_data)

#save same dataset for later graph (jitter plot)
jitter_data <- plot_data

#Create violin plot
#How to create a violin plot was referenced from here: https://www.sthda.com/english/wiki/ggplot2-violi
violin_plot <- ggplot(plot_data, aes(x = prior_malignancy, y = count_value, fill = prior_malignancy)) +
  geom_violin() +
  scale_fill_brewer(palette = "Pastel1") +
  labs(title = paste("Violin Plot for", first_gene_name,"Gene"),
       x = "Prior Malignancy",
       y = "Gene Count") + theme(plot.title = element_text(size = 15, hjust = 0.5, face = "bold"),
                                 legend.position = "none", axis.title = element_text(face = "bold"))

print(violin_plot)
```



3) Updated heatmap improved with the feedback from your second submission.

```
#Heatmap

#Data prep

#import all necessary libraries
library(tidyr)
library(dplyr)
library(stringr)
library(circlize)
```

```
## ========================================
## circlize version 0.4.16
## CRAN page: https://cran.r-project.org/package=circlize
## Github page: https://github.com/jokergoo/circlize
## Documentation: https://jokergoo.github.io/circlize_book/book/
##
## If you use it in published research, please cite:
## Gu, Z. circlize implements and enhances circular visualization
##   in R. Bioinformatics 2014.
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(circlize))
## ========================================
```

```r
library(ComplexHeatmap)
```

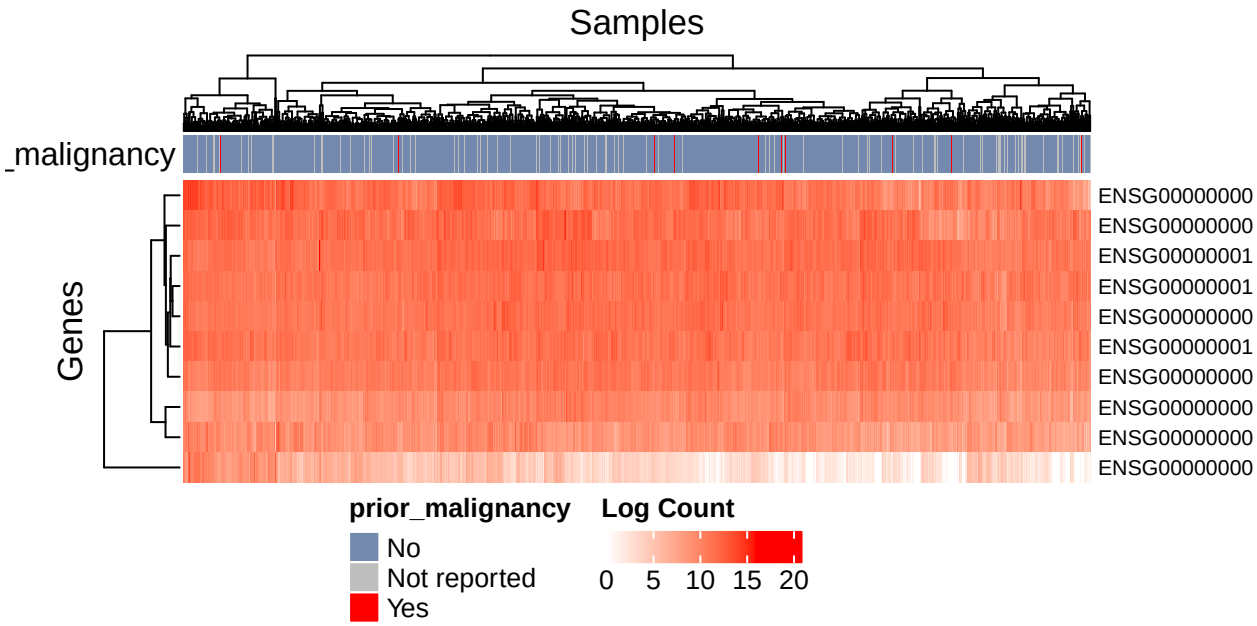
```
## Loading required package: grid
```

```
## ========================================
## ComplexHeatmap version 2.24.1
## Bioconductor page: http://bioconductor.org/packages/ComplexHeatmap/
## Github page: https://github.com/jokergoo/ComplexHeatmap
## Documentation: http://jokergoo.github.io/ComplexHeatmap-reference
##
## If you use it in published research, please cite either one:
## - Gu, Z. Complex Heatmap Visualization. iMeta 2022.
## - Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional
##     genomic data. Bioinformatics 2016.
##
##
## The new InteractiveComplexHeatmap package can directly export static
## complex heatmaps into an interactive Shiny app with zero effort. Have a try!
##
## This message can be suppressed by:
##   suppressPackageStartupMessages(library(ComplexHeatmap))
## ========================================
```

```r
#heatmap code from lecture 3 notes and referenced from tutorial here: https://biostatsquid.com/heatmaps

#get 10 genes
genes_to_plot <- rownames(counts)[1:10]
counts_subset <- counts[genes_to_plot, ]
#head(counts_subset)

#filter for the fist gene and rename first column for gene name

names(counts_subset)[1] <- "gene_name"
#head(counts_subset)

#Use pivot_longer function from lecture to pivot data for analysis, different method learned here: #htt
subset_gene.dt <- counts_subset %>%
  pivot_longer(-gene_name, names_to = "sample", values_to = "count_value")

#head(subset_gene.dt)

#Get metadata
metadata <- read.csv(file = "meta_data.csv", stringsAsFactors = FALSE)
#head(metadata)

#use gsub function to replace "." with "-" in the values of the sample column, so join will work downst

subset_gene.dt$sample <- gsub("\\.","-", as.character(subset_gene.dt$sample))
#head(subset_gene.dt)

#Change sample column to barcode, to match name in metadata for easy join
names(subset_gene.dt)[names(subset_gene.dt) == "sample"] <- "barcode"
#head(subset_gene.dt)

# Join subset_gene.dt for my selected genes to the metadata dataset, by the barcode column, to bring in
```

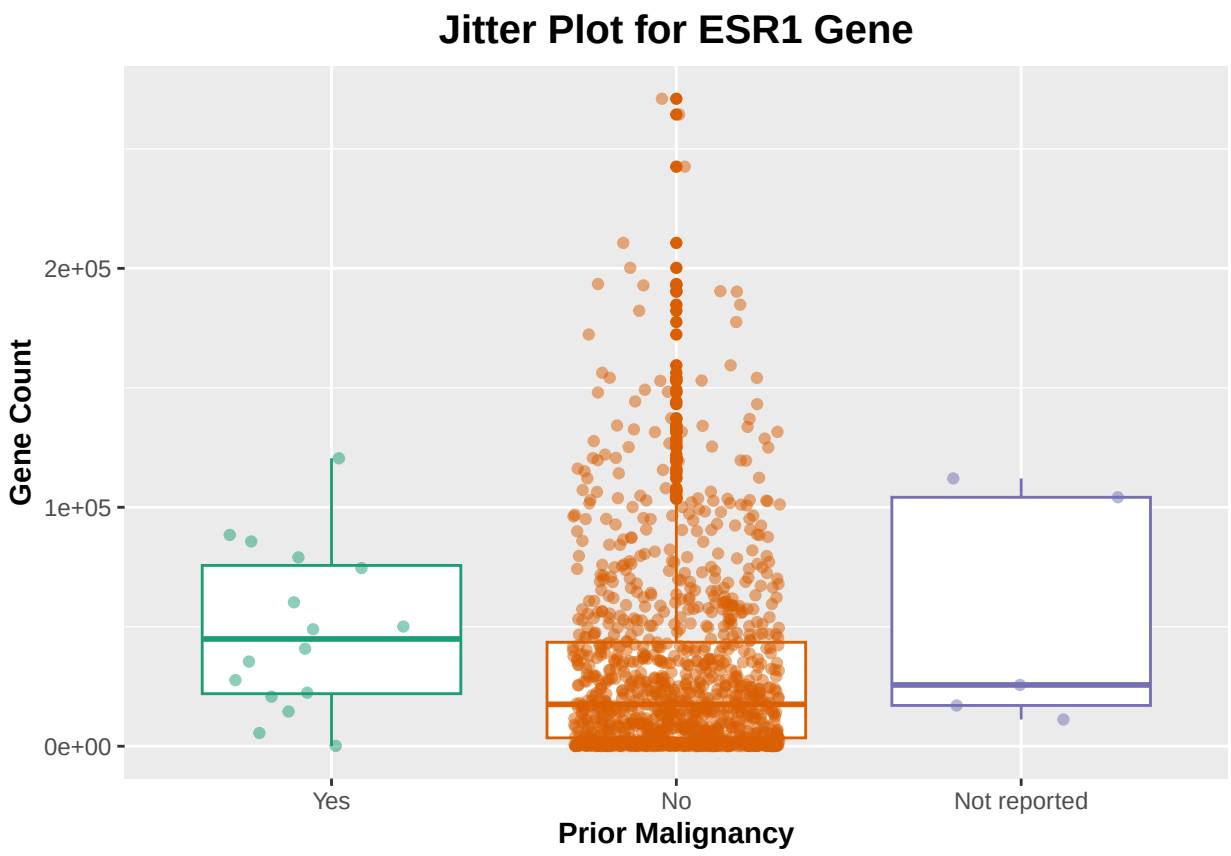
```r
plot_data <- subset_gene.dt %>%
  inner_join(select(metadata, barcode, prior_malignancy),
             by = "barcode")

#print(plot_data)

#Convert prior_malignancy to camelcase, using str_to_sentence from here: https://stringr.tidyverse.org/
plot_data$prior_malignancy <- str_to_sentence(plot_data$prior_malignancy)

#head(plot_data)

#change data structure
heatmap_data <- as.matrix(
  xtabs(count_value ~ gene_name + barcode, data = plot_data)
)

#get log of data for heatmap visualization
heatmap_data_log <- log2(heatmap_data + 1)

## Heatmap annotations
annotation <- plot_data %>%
  distinct(barcode, prior_malignancy)


column_ha <- HeatmapAnnotation(
  prior_malignancy = annotation$prior_malignancy[match(colnames(heatmap_data_log), annotation$barcode)],
  col = list(
    prior_malignancy = c("Yes" = "red", "No" = "#7389AE", "Not reported" = "gray")),
  annotation_name_side = "left"
  )

#define color for legend
col_fun <- colorRamp2(c(min(heatmap_data_log), max(heatmap_data_log)), c("white", "red"))

# Create heatmap
gene_heatmap <- Heatmap(
  heatmap_data_log,
  #name = "Log Count",
  show_heatmap_legend = FALSE,
  col = col_fun,
  top_annotation = column_ha,
  show_row_names = TRUE,
  show_column_names = FALSE,
  cluster_columns = TRUE,
  cluster_rows = TRUE,
  column_title = "Samples",
  row_title = "Genes",
  width = unit(12, "cm"),
  height = unit(4, "cm"),
  row_names_gp = gpar(fontsize = 8),
  column_names_gp = gpar(fontsize = 6)
)
```

```
# Create legend using lgd function from https://jokergoo.github.io/ComplexHeatmap-reference/book/legend
lgd <- Legend(col_fun = col_fun, title = "Log Count", direction = "horizontal")

library(grid)
draw(gene_heatmap,
     annotation_legend_list = list(lgd),
     annotation_legend_side = "bottom",
     padding = unit(c(4, 4, 10, 4), "mm"))
```



4) Take some time to Google around and generate an additional new plot type is not yet included in your report. This plot can be one we learned about in class.

```
#Create jitter plot & box plot
#tutorial found here: https://www.cedricscherer.com/2019/08/05/a-ggplot2-tutorial-for-beautiful-plottin

jitter_plot <-
  ggplot(jitter_data, aes(x = prior_malignancy, y = count_value,
                  color = prior_malignancy)) +
    labs(title = paste("Jitter Plot for", first_gene_name,"Gene"),
      x = "Prior Malignancy", y = "Gene Count") +
       scale_color_brewer(palette = "Dark2", guide = "none") +
  theme(plot.title = element_text(size = 15, hjust = 0.5, face = "bold"),
        axis.title = element_text(face = "bold"),
        legend.position =  "none")

jitter_plot + geom_boxplot() + geom_jitter(width = .3, alpha = .5)
```


Jitter Plot for ESR1 Gene
Gene Count
2e+05
1e+05
0e+00
Yes
No
Not reported
Prior Malignancy